

Directed Dynamic Symbolic Execution for Static Analysis Warnings Confirmation

A. Yu. Gerasimov^{a,*}

^a*V.P. Ivannikov Institute for System Programming, Russian Academy of Sciences,
ul. Solzhenitsyna 25, Moscow, 109004 Russia*

^{*}*e-mail: agerasimov@ispras.ru*

Received May 16, 2018

Abstract—Currently, there is no doubt among experts in the field of program certification and quality assurance that automated program analysis methods should be used to find bugs that lead to program security vulnerabilities. The national standard for the secure software development requires the use of source code static analysis tools as one of the measures of software quality assurance at the development stage and the application of dynamic analysis and fuzz-testing of the source code at the qualification testing stage. Fundamental limitations of automated program analysis and testing methods make it impossible to carry out simultaneously exhaustive and precise analysis of programs for errors. Thereof, researches are nowadays carried out aimed at reducing the effect of fundamental limitations on the quality and productivity of automated software error detection methods. This paper discusses an approach that combines methods of source code static analysis and dynamic symbolic execution in order to increase the program error detection efficiency.

DOI: 10.1134/S036176881805002X

1. INTRODUCTION

The development of methods and the use of automated source code analysis tools for error detection have become widespread since the early 2000s [1–3]. The National Standard of the Russian Federation *GOST R 56939–2016: Secure Software Development* establishes the requirements for the use of source code static analysis tools as one of the measures to ensure the quality of software at the stage of the program design and building (Clause 5.3.3.4 *GOST R 56939–2016*): “The software developer shall ensure that the source code is statically analyzed in order to identify program deficiencies (potentially vulnerable statements) in the source code. Static analysis of the source code should be carried out with respect to components borrowed from third-party software developers if the source code of these components is available. Based on the source code static analysis results, the program can be refined.” [4]. At the same time, the fundamental limitations [5], as well as a permanent trade-off between performance and precision of static analysis of programs [6], lead to false positives, the number of which is often quite large. In turn, low analysis precision, as well as the high complexity of the process of verification of the results of automated tools, lead to the fact that software developers refuse to use source code automated analysis tools [7, 8].

On the other hand, the methods of program dynamic analysis in accordance with *GOST R 56939–2016*

should be used to identify program vulnerabilities (Clause 5.4.1 *GOST R 56939–2016*). Program dynamic analysis methods make it possible to detect the erroneous behavior of programs with high precision due to the fact that bugs are identified in the process of program execution and provide input data that can be used to reproduce the erroneous behavior of the program. At the same time, program dynamic analysis methods have low error detection performance due to the path explosion problem during an exhaustive analysis of the program [9].

As a result, the problem of combining various program analysis methods arises in order to overcome the above-mentioned limitations that prevent their separate use in industry at different stages of the software life cycle. Moreover, the use of agile development processes in both small teams and large corporations that develop software products involves the use of *continuous integration* of the software product and makes it possible to apply static and dynamic analysis methods directly at the stage of program design and building.

This paper proposes an approach to combining methods of source code static analysis and dynamic symbolic execution to improve the analysis precision and performance. The paper is structured as follows: Section 2 gives an overview of the domain, Section 3 describes the proposed approach, Section 4 presents results of applying the proposed approach to a bundle of open source projects, Section 5 compares the proposed method with similar methods, and the Conclu-

sions section provides an overview of further research directions.

2. OVERVIEW OF PROGRAM ANALYSIS METHODS

2.1. Error Representation in Static Analysis

Program defect detection tools are of great interest among modern source code static analysis tools. These tools can detect both a noncompliance with simple rules or standards for source code style, such as some MISRA [10] or CERT [11] rules and critical runtime errors that can lead to either a crash or to user data corruption or leaks. Bugs that can become a source of security vulnerabilities in the program, allow unauthorized access to user data, execute payload, or cause a crash are the point of great interest for the developers, software quality assurance specialists, and certification organizations. Such bugs are usually represented as a sequence of events in the program from the source point to the sink point. Listing 1 shows a program fragment with an error that leads to the use of deallocated memory, which can result in a crash.

Listing 1: a

```
char *p, *pos;
p = malloc(size);
// Triggering event
if (size > 0) {
    ... // do some work
    // Source point
    free(p);
}
// Sink point
pos = strstr(p, buffer);
```

In this example, the source point is the call of the *free* function for the pointer variable *p*, the sink point is the *strstr* function call site, and the event that is important for the error is the condition of entering the then-block of the conditional statement. The sequence of events will be called the *error trace*, the point at which the trace begins is called the *source point*, and the point at which the trace ends is called the *sink point* or the *error point*. The error trace must contain at least a sink point. The source point and the sequence of events leading to the error are optional. For example, for the “Use of Deprecated Function” error (for example, functions from outdated program libraries or compromised encryption algorithms), the checker of which is designed to search for calls of functions that are not recommended to use, there are no source point and event points in the program that are important for the error realization, since the actual sink point will be the only point in the trace.

In the process of program analysis, the static analyzer searches for the execution paths from the source point to the sink point with the checks for consistency

of conditions on the given path. However, due to the scalability and high analysis performance requirements, static analysis tools usually do not check the full program execution path from the entry point containing the error trace, and this can lead to false positives.

The *source–sink* error class includes a quite large number of different errors in the program, many of which can lead to the user data leak, payload execution, or crash:

- Uninitialized or null pointer dereference.
- Buffer bounds violation on the stack or heap.
- Memory and other resource leaks.
- Integer overflow.

2.2. Dynamic Symbolic Execution

Dynamic symbolic execution of programs was originally developed as a test coverage generation method [9, 12]. However, since the middle 2000s, there has been an explosive growth of tools for analyzing programs based on this method [18–23]. There are *online* and *offline* (or *postmortem*) program analyses using dynamic symbolic execution. In the process of *online* analysis, at every branch condition point, a new process spawns to follow the other branch. In the process of *offline* analysis, program iterative execution is performed (each time with new input data leading to program execution along a new path). Algorithm 1 shows the sequence of actions when performing *offline* dynamic symbolic analysis.

Algorithm 1. Offline dynamic symbolic execution

```
inputs ← seed
repeat
    cur_input ← getBestInput(inputs)
    trace ← traceExecution(cur_input)
    smt_formula ← convert(trace)
    smt_formulas ← split(smt_formula)
    for ∀smt_formula ∈ smt_formulas do
        inputs ← estimate(smt_formula)
    end for
until |inputs| ≠ ∅
```

Dynamic symbolic execution starts with putting the seed input to the input collection. Each *offline* iteration of dynamic symbolic execution starts the selection of the best-known program input by the *getBestInput* function. Next, the *traceExecution* function executes the program with the selected input and writes the program *trace* for its subsequent conversion to the path constraint in the form of the *smt_formula* for Satisfiability Modulo Theories (SMT) solvers [13, 14] using the *convert* function. Next, the *smt_formula* is split into several *smt_formulas* by the number of

branch points, the direction of which should be inverted for the next iteration of dynamic symbolic execution, after which the input data are calculated (if formula constraints can be satisfied) and the input obtained is evaluated. The algorithm continues until all inputs for feasible program paths have been exhausted or, if it is known that the program can be executed on an infinite input set, up to the forced stop of the algorithm execution when the time limit or some other criterion is reached.

A numerical metric, with the help of which the paths are prioritized according to analysis goals, is used to evaluate the inputs. For example, if the goal of the analysis is to maximize the fast growth of coverage by the basic blocks of the program during the analysis, then the number of new basic blocks of the program through which the program will be executed when this input is fed to the program input can be used as such a metric. The following heuristics are also used [15]:

- random selection of a new input;
- depth-first search, the input that makes it possible to analyze the longest available program path;
- breadth-first search, the input that first examines all the directions of topmost branches in the program;
- preference for paths with memory access by symbolic address [16];
- preference for paths with detected errors that did not lead to the crash (buggy-path first) [17], etc.

2.3. Directed Dynamic Symbolic Execution of Programs

The path explosion problem in the process of dynamic symbolic execution was identified back in 1975 [9] as a fundamental limitation for the practical use of the dynamic symbolic execution method to the exhaustive program analysis. In this regard, different groups of researchers have searched for the ways to use the dynamic symbolic execution method for solving various practical problems. The work [24] describes the approach of applying directed dynamic symbolic execution to the generation of regression tests only for the modified program part. The essence of the proposed approach is to prune paths that do not pass through the modified part of the program. A similar approach is used in [25] to solve the same problem.

The work [26] describes an approach is proposed consisting in using dynamic symbolic execution to compute the input data that lead to program execution at the point at which the error occurred. A crash stack trace is used as the initial information. The tool constructs input data for the program in such a way as to pass through all points reflected in the crash stack trace. To do this, a preliminary static analysis of the program is performed to calculate intermediate nodes (elements of the call stack at the time of crash) along the interprocedural control-flow graph, which should lie on the path leading to the program crash. Further, dynamic symbolic execution is used to follow a

selected path along which the distance (proximity) to the next point in the crash stack is minimal with respect to the number of processor instructions to be executed.

In [27], a similar approach is used to solve the problem of constructing program input data which will be used to execute a certain source code line. Distances from each graph node to the node with the point of interest in the program are precalculated based on the program's interprocedural control-flow graph. Next, two alternative approaches are used to find the path in the program that leads to the point of interest: dynamic symbolic execution that uses the distance from the execution path to the point of interest in the program as the prioritization metric of possible execution paths and dynamic symbolic execution applied to individual functions in the order reverse to the normal chain of program calls (call-chain backward symbolic execution). A similar approach for reaching the point of interest in the program but already in the case of *online* dynamic symbolic execution is described in [28].

3. DYNAMIC SYMBOLIC EXECUTION DIRECTED BY THE STATIC PROGRAM ANALYSIS RESULTS

As noted in [29], false positives in program defect detection can be caused by insufficient sensitivity to the context of the function call due to a compromise between the precision and performance of the static analysis tool. Consequently, the problem of improving quality of static program analysis results comes to existence, which is solved by various methods [30]. Among them, we note guided (directed) dynamic symbolic execution to calculate input data of the program, which can be used to demonstrate an error and debug the program. The approach described in this section is based in part on results of the previous study [31].

The initial data for the problem being solved consist of

- (1) error warning from the static analyzer and
- (2) executable code with debug info corresponding to the source code analyzed by the static analyzer.

The solution of the problem will provide either an input indicating a program defect, or confirmation that conditions on the error trace are inconsistent, or a message about the impossibility neither to confirm nor to deny that there is a program defect because of the limitations of the proposed approach [32].

The solution is divided into several stages:

- Static analysis of the executable code to obtain a slice of the program's interprocedural control-flow graph leading to program execution along paths with the error trace, to evaluate each program basic block as the distance from the basic block on the error path to the error trace event points in the form of the number of branch points between basic block and the points.

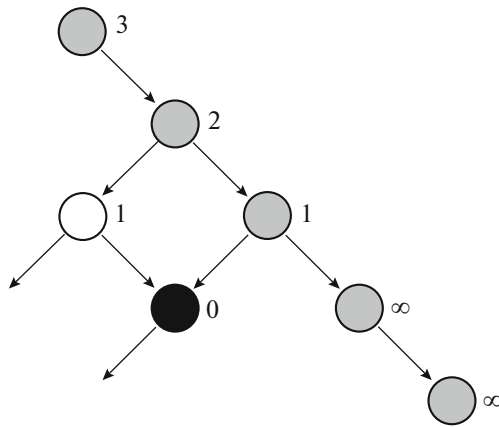


Fig. 1. Program transition graph.

- Iterative offline dynamic symbolic execution of the program with the execution path distance metric enabled from the next point on the error trace until the program is executed through the sink point to prioritize the input.
- Adding asserts for security predicate violation, which indicate program defects, to the path condition.

3.1. Executable Source Code Static Analysis

The task of the first stage is to obtain a slice of the program's control-flow graph with the evaluation of the distance of each basic block from the next point on the error trace. The path comparison algorithm is divided into several subtasks:

1. Construction of a reduced program call graph that describes all the paths to reach the function that contains the sink point. It is carried out by the backward pass method from the function containing the sink point to the functions that call it and so on, up to the entry point to the program or thread function.

2. Construction of a reduced control-flow graph for functions from the reduced program call graph constructed on stage 1 of the algorithm. All nodes and transition edges for basic blocks that are not on the possible execution paths containing the error trace are removed from the function control flow graph. An important part of the first analysis stage is matching of the path in the source code with the path in the executable code. As noted in [31], the number of basic blocks in the source and executable code differ if complex conditional expressions are used. Listing 2 shows an example of a C program fragment that contains two basic blocks: *then-block* and *else-block*. However, when the program is compiled into the executable code, the basic blocks are added for the computation of the conditional expression of the branch point operator, so that the number of basic blocks for the specified fragment in the executable code is four.

Listing 2: Example of evaluating a conditional expression

```

if (A || B) { ... } else { ... }
4004 f6: cmp $0x0, -0x4(%rbp)
4004 fa: jne 400502; A
4004 fc: cmp $0x0, -0x3(%rbp)
400500: je 400515; B
400502: ; then-branch
...
400515: ; else-branch

```

The result of static analysis of the executable code is the comparison map of addresses of the basic blocks with the distance to the next error trace event on the execution path graph.

3.2. Directed Dynamic Symbolic Execution of Program

We use *offline* dynamic symbolic execution of programs, with the minimum distance to the next point on the error trace being the path preference metric, to calculate the input data of the program and show errors in our approach. Figure 1 shows a fragment of the reduced control-flow graph of the program containing the error point in the program (black node). Nodes that were used at the previous iteration of the dynamic symbolic execution are paint in gray. It is required to select one of the paths for the next iteration of the dynamic symbolic execution. There are two alternative unexplored paths in the program with the estimates for the distance to the sink points equal to 2 and 1. Accordingly, the path with the minimal estimate 1 will be selected.

Thus, in the course of directed dynamic symbolic execution of the program, the search of feasible execution paths with the program error trace is carried out.

3.3. Security Predicate Violation Check

The final stage in checking the program error realization is to check the security predicate violation. If the path constraint formula with asserts for security predicate violation for a certain type of error can be computed, the solver returns values of the free variables of the formula that actually represent the new input data for the program that lead to the potentially dangerous operation to uncover erroneous behavior if they fed to the program as input.

For each error class, it is required to define a security predicate, the violation of which will lead to error. In [33], an example of security predicates is given for **divide by zero** and **null pointer dereference errors**. In order to check the program security violation for the division operation, it necessary to check the divisor computability to zero. In order to check for a program security violation for the null-pointer dereference, it is necessary to check the computability of the *NULL* pointer value. The work [34] describes security

predicates for errors associated with **return address substitution** and **function pointer substitution** as well as control flow interception by **attacking the format string function**. In [36], an example of a security predicate description for the **buffer overflow (bounds violation)** error is given, which consists in monitoring lengths of the buffer and data that is written to it.

For a vast class of errors, it is sufficient to check the possibility of execution along a certain path in the program. For example, for **resource leak** errors, such as **dynamically allocated memory leak**, and **undeallocated operating system resources**, such as files, sockets, etc., the error is confirmed by traversing the path on which there is operating system resource allocation function call but no deallocation function call. In [35], an algorithm is presented for checking warnings of the *Fortify* static analyzer [37] related to undeallocated dynamic memory using dynamic symbolic execution implemented by the *CREST* tool [38], which consists in checking the feasibility of all program execution paths on which a memory leak can occur. **Use after free** and **dangling pointer** errors also belong to this class. In [39], the detection of the synchronization error of file write and rename operations, which is also confirmed by execution along the path containing the error trace.

It should be noted that the security predicate violation does not always lead to a crash. For example, if violation of bounds of a buffer allocated in the dynamic memory does not violate memory page bounds allocated to the program by the operating system, then **segmentation fault** or **access violation** runtime errors do not occur. Such cases are considered a silent crash [40], when the program execution continues, but, at the same time, the values of the memory cells on the heap or on the stack are damaged.

4. EXPERIMENTAL EVALUATION OF THE PROPOSED APPROACH

The proposed approach was experimentally evaluated based on the *Svace* source code static analysis tool [41] and the *Anxiety* dynamic symbolic execution tool [42] developed at the Ivannikov Institute for System Programming of Russian Academy of Sciences. For the *Svace* tool, a module was developed that makes it possible to save positions of branch points and basic blocks for *then*- and *else*-branches for further conversion of warnings in the form of addresses in the executable code convenient for further use in the dynamic symbolic execution process. For the *Anxiety* tool, a directed analysis module was developed that preliminarily performs a static analysis of the executable code in order to extract reduced call and control flow graphs for paths containing error traces and further prioritizes execution paths in accordance with the distance metric to the next point on the error trace, for directing the dynamic symbolic execution, in order to traverse the path containing the entire error trace.

Table 1. Buffer overflow errors

Overflow		
Error type	SA	DA
Buffer	94	59
After Check	5	2
Under Check	52	0
String	32	26
Static	1	0
Total	184	87

The proposed approach was evaluated on a set of command-line utilities from the Debian Linux operating system:

- **a2ps** is an Any to PostScript filter;
- **a56** is an assembler for Motorola DSP56001;
- **cabextract** is a Microsoft Cabinet file unpacker;
- **dvips** is a TeX-to-PostScript converter;
- **elfutils** is a utility for unpacking and displaying the contents of ELF files;
- **epstool** is a PostScript and PDF interpreter;
- **expat** is a stream-oriented XML parser;
- **faad2** is a decoder of audio files;
- **jasper** is a graphic file converter;
- **mp3val** is a utility for audio files validation;
- **orcc** is a utility for creating containers for different data types;
- **pngtools** is a utility for displaying information about a PNG file;
- **unmo3** is a utility for converting music files;
- **yodl** is a document conversion utility.

The analysis of the results obtained by error types is given below. The *SA* column in the tables corresponds to the number of warnings about errors detected by static analysis methods, and the *DA* column corresponds to the number of error traces traversed by directed dynamic symbolic execution.

The tool was able to reproduce error traces associated with overflows of buffers of different types in 47% of cases (Table 1). This result does not answer the question of whether all reproduced traces are true errors or false positives, since they require the use of specific security predicates to confirm out-of-bound access to allocated memory buffers.

The tool was able to reproduce error traces associated with dereferencing of null or freed pointer in 48% of cases (Table 2). For all types of errors, except for **Deref of NULL**, it is sufficient to pass along the error path to check the error.

The tool was able to reproduce the error traces associated with the reuse of the freed object in 40% of cases (Table 3). For all types of errors, it is sufficient to pass along the error path to check the error.

Table 2. Dereference of NULL

Dereference		
Error type	SA	DA
Deref after Null	114	41
NULL after deref	26	22
Deref of NULL	116	61
Total	256	124

Table 3. Use after free

Use after release		
Error type	SA	DA
Double close	17	6
Use after release	2	2
Double free	5	2
Use after free	3	1
Deref after free	5	2
Total	32	13

Table 4. Resource leak

Resource leak		
Error type	SA	DA
Handle leak	22	1
Memory leak	124	13
Dangling pointer	378	261
Total	524	275

The tool was able to reproduce the error traces associated with resource leak in 52% of cases (Table 4). For all types of errors, it is sufficient to pass along the error path to check the error.

Analysis of Results. It should be noted that the currently implemented method does not use security predicates to check for various errors that require additional checks, such as buffer overflow evaluation or null-pointer dereference, which adversely affects the error confirmation precision.

The negative effect of the conservative approach to tracking the symbolic data flow of the program in the process of symbolic execution, also called undertaintness of the data flow of the program, should also be noted. Currently, only the operations that use symbolic variables (corresponding to input data or generated from them) as operands are used in symbolic formulas corresponding to the operations over symbolic data of the program. At the same time, the static analyzer can take into account the values of variables that indirectly depend on the input data, for example, the ones obtained by tabular conversions using a symbolic

index, by initializing and/or changing the value under a condition depending on the symbolic variables or with the jump to the calculated address.

In addition, the presented method does not describe the directed dynamic symbolic execution of programs specially protected from analysis by similar methods.

5. COMPARISON WITH ANALOGS

One of the early and close analogs of the proposed approach is DyTa tool [44], which uses path pruning to avoid dynamic symbolic execution of paths that do not lead to the point of error realization for errors found by the static analyzer of C# programs. An alternative approach is used in the SANTE tool [45], which combines the *Frama-C* static program analyzer in C (it marks the points of detection of potential run-time errors in the program and carries out slicing (division into layers) so that only operations on paths associated with the detected errors remained in the program), and the PathCrawler tool for dynamic symbolic execution [47], which automatically bypasses paths remaining in the program after the slicing in order to generate an input for error confirmation. The work [35] also employs the approach of confirming errors found by static analysis with the help of dynamic analysis, but only for memory leak errors.

6. CONCLUSIONS

The paper presents intermediate research results in the field of directed dynamic symbolic execution used to calculate input data for processing a specific set of addresses of program instructions corresponding to points on the error trace found by the source code static analyzer. The effectiveness of the proposed approach is evaluated based on the prototype of the program system for source code static analysis and directed dynamic symbolic execution of programs. To improve accuracy of error confirmation, we plan to study

(1) security predicates for various types of errors and their use on the last stage of the directed dynamic symbolic execution for error implementation confirmation;

(2) solution of the problem of sufficient taintedness of program data flow in order to resolve the problem of symbolic data flow break caused by indirect data and control dependences.

REFERENCES

1. Ayewah, N., Hovemeyer, D., Morgenthaler, J.D., Penix, J. and Pugh, W., Experiences using static analysis to find bugs, *IEEE Software*, 2008, vol. 25, no. 5, pp. 22–29.
2. Calcagno, C., Distefano, D., Dubreil, J., Gabi, D., Hooimeijer, P., Luca, M., O'Hearn, P.W., Papakon-

- stantinou, I., Purbrick, J., and Rodriguez, D., Moving fast with software verification, *Lect. Notes Comput. Sci.*, 2015, vol. 9058, pp. 3–11.
3. Calcagno, C. and Distefano, D., Infer: An automatic program verifier for memory safety of C programs, *Lect. Notes Comput. Sci.*, 2011, vol. 6617, pp. 459–465.
4. GOST R 56939-2016: Information protection. Secure software development. General requirements. — National Standard of Russian Federation, 2016
5. Turing, A.M., On computable numbers, with an application to the Entscheidungsproblem, *Proc. London Math. Soc.*, 1937, vol. s2–42, no. 1, pp. 230–265.
6. Belevantsev, A.A., Multilevel static analysis of source code for program quality assurance, *Doctoral (Phys.–Math.) Dissertation*, Moscow, 2017.
7. Johnson, B., Song, Y., Murphy-Hill, E., and Bowdidge, R., Why don't software developers use static analysis tools to find bugs? *Proc. of the 2013 Int. Conf. on Software Engineering*, San Francisco, 2013, pp. 672–681.
8. Sadowski, C., Aftandilian, E., Eagle, A., Miller-Cushon, L., and Jaspan, C., Lessons from building static analysis tools at Google, *Commun. ACM*, 2018, vol. 61, no. 4, pp. 58–66.
9. Boyer, R.S., Elspas, B., and Levitt, K.N., SELECT—a formal system for testing and debugging programs by symbolic execution, *Proc. of the Int. Conf. on Reliable software*, Los Angeles, 1975, pp. 234–245.
10. *Guidelines for the Use of the C Language in Critical Systems*, Warwickshire: MISRA, 2013.
11. Seacord, R.C. and Martin, R., *MITRE CWE and CERT Secure Coding Standards*, Boston: Addison-Wesley Professional, 2009.
12. King, J.C., Symbolic execution and program testing, *Commun. ACM*, 1976, vol. 19, no. 7, pp. 385–394.
13. Ganesh, V. and Dill, D.L., A decision procedure for bit-vectors and arrays, *Proc. 19th Int. Conf. on Computer Aided Verification*, Berlin, 2007, pp. 519–531.
14. De Moura, L., and Bjorner, N., Z3: An efficient SMT solver, *Lect. Notes Comput. Sci.*, 2008, vol. 4963, pp. 337–340.
15. Baldoni, R., Coppà, E., D'Elia, D.C., Demetrescu, C., and Finocchi, I., A survey of symbolic execution techniques, *ACM Comput. Surv.*, 2018, vol. 51, no. 3.
16. Cha, S.K., Avgerinos, T., Rebert, A., and Brumley, D., Unleashing mayhem on binary code, *Proc. 2012 IEEE Symp. on Security and Privacy*, San Francisco, 2012, pp. 380–394.
17. Avgerinos, T., Cha, S.K., Rebert, A., Schwartz, E.J., Woo, M., and Brumley, D., Automatic exploit generation, *Commun. ACM*, 2014, vol. 57, no. 2, pp. 74–84.
18. MacIntyre, B., Gandy, M., Dow, S., and Bolter, J.D., DART: A toolkit for rapid design exploration of augmented reality experiences, *Proc. 17th Annual ACM Symp. on User Interface Software and Technology*, Santa Fe, 2004, pp. 197–206.
19. Sen, K., Marinov, D., and Agha, G., CUTE: A concolic unit testing engine for C, *Proc. 10th European Software Engineering Conf. held jointly with 13th ACM SIGSOFT Int. Symp. on Foundations of Software Engineering*, Lisbon, 2005, pp. 263–272.
20. Cadar, C., Ganesh, V., Pawlowski, P.M., Dill, D.L., and Engler, D.R., EXE: Automatically generating inputs of death, *ACM Trans. Inform. Syst. Secur.*, 2008, vol. 12, no. 2.
21. Godefroid, P., Levin, M.Y., and Molnar, D., Automated whitebox fuzz testing, *Proc. 16th Annual Network & Distributed System Security Symp.*, San Diego, 2008, pp. 151–166.
22. Cadar, C., Dunbar, D., and Engler, D., KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, *Proc. 8th USENIX Conf. on Operating Systems Design and Implementation*, San Diego, 2008, pp. 209–224.
23. Chipounov, V., Kuznetsov, V., and Candea, G., S2E: A platform for in-vivo multi-path analysis of software systems, *Proc. 16th Int. Conf. on Architectural Support for Programming Languages and Operating Systems*, Newport Beach, 2011, pp. 265–278.
24. Taneja, K., Xie, T., Tillmann, N., de Halleux, J., and Schulte, W., Guided path exploration for regression test generation, *Proc. 2011 Int. Symp. on Software Testing and Analysis*, Toronto, 2011, pp. 1–11.
25. Yang, G., Person, S., Rungta, N., and Khurshid, S., Directed incremental symbolic execution, *Proc. 32nd ACM SIGPLAN Conf. on Programming Language Design and Implementation*, San Jose, 2011, pp. 504–515.
26. Zamfir, C. and Candea, G., Execution synthesis: A technique for automated software debugging, *Proc. 5th European Conf. on Computer Systems*, Paris, 2010, pp. 321–334.
27. Ma, K.K., Phang, K.Y., Foster, J.S., and Hicks, M., Directed symbolic execution, *Proc. 18th Int. Conf. on Static Analysis*, Venice, 2011, pp. 95–111.
28. Parvez, R., Ward, P.A.S., and Ganesh, V., Combining static analysis and targeted symbolic execution for scalable bug-finding in application binaries, *Proc. 26th Annual Int. Conf. on Computer Science and Software Engineering*, Toronto, 2016, pp. 116–127.
29. Borodin, A.E., Interprocedural context-sensitive static analysis for finding defects in C/C++ program source code, *Cand. Sci. (Phys.–Math.) Dissertation*, Moscow, 2016.
30. Gerasimov, A.Yu., Survey on static program analysis results refinement approaches, *Tr. Inst. Sistemnogo Program. Ross. Akad. Nauk*, 2017, vol. 29, no. 3, pp. 75–98.
31. Gerasimov, A.Yu., Kruglov, L.V., Ermakov, M.K., and Vartanov, S.P., An approach of reachability determination for static analysis defects with help of dynamic symbolic execution, *Tr. Inst. Sistemnogo Program. Ross. Akad. Nauk*, 2017, vol. 29, no. 5, pp. 111–134.
32. Gerasimov, A.Yu., On limitations of the classification method of program defects found by static program analysis methods using dynamic symbolic execution, *Trudy 60-i Vserossiiskoi nauchnoi konferentsii MFTI (Proc. 60th MPTI All-Russian Scientific Conf.)*, Moscow, 2017.
33. Isaev, I.K. and Sidorov, D.V., The use of dynamic analysis for generation of input data that demonstrates critical bugs and vulnerabilities in programs, *Program. Comput. Software*, 2010, vol. 36, no. 4, pp. 225–236.
34. Fedotov, A.N., Kaushan, V.V., Gaisaryan, S.S., and Kurmangaleev, Sh.F., Building security predicates for

- some types of vulnerabilities, *Tr. Inst. Sistemnogo Program. Ross. Akad. Nauk*, 2017, vol. 29, no. 6, pp. 151–162.
35. Li, M., Chen, Y., Wang, L., and Xu, G., Dynamically validating static memory leak warnings, *Proc. 2013 Int. Symp. on Software Testing and Analysis*, Lugano, 2013, pp. 112–122.
 36. Xu, R.-G., Godefroid, P., and Majumdar, R., Testing for buffer overflows with length abstraction, *Proc. 2008 Int. Symp. on Software Testing and Analysis*, Seattle, 2008, pp. 27–38.
 37. Fortify Analysis Software. https://software.microfocus.com/en-us/solutions/application-security?jumpid=va_912rztnd7/. Cited May 15, 2018.
 38. Burnim, J. and Sen, K., Heuristics for scalable dynamic test generation, *Proc. 2008 23rd IEEE/ACM Int. Conf. on Automated Software Engineering*, 2008, pp. 443–446.
 39. Cui, H., Hu, G., Wu, J., and Yang, J., Verifying systems rules using rule-directed symbolic execution, *Proc. 18th Int. Conf. on Architectural Support for Programming Languages and Operating Systems*, Houston, 2013, pp. 329–342.
 40. Neugschwandtner, M., Comparetti, P.M., Haller, I., and Bos, H., The BORG: Nanoprobing binaries for buffer overreads, *Proc. 5th ACM Conf. on Data and Application Security and Privacy*, San Antonio, 2015, pp. 87–97.
 41. Ivannikov, V.P., Belevantsev, A.A., Borodin, A.E., Ignat'ev, V.N., Zhurikhin, D.M., Avetisyan, A.I., and Leonov, M.I., Static analyzer Svace for finding defects in a source program code, *Tr. Inst. Sistemnogo Program. Ross. Akad. Nauk*, 2014, vol. 40, no. 5, pp. 265–275.
 42. Gerasimov, A., Vartanov, S., Ermakov, M., Kruglov, L., Kutz, D., Novikov, A., and Asryan, S., Anxiety: A dynamic symbolic execution framework, *Proc. 2017 Ivannikov ISPRAS Open Conf.*, Moscow, 2017, pp. 16–22.
 43. Schwartz, E.J., Avgerinos, T., and Brumley, D., All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask), *Proc. 2010 IEEE Symp. on Security and Privacy*, Oakland, 2010, pp. 317–331.
 44. Ge, X., Taneja, K., Xie, T., and Tillmann, N., DyTa: Dynamic symbolic execution guided with static verification results, *Proc. 33rd Int. Conf. on Software Engineering*, Honolulu, 2011, pp. 992–994.
 45. Chebaro, O., Kosmatov, N., Giorgetti, A., and Juliand, J., The sante tool: Value analysis, program slicing and test generation for C program debugging, *Proc. 5th Int. Conf. on Tests and proofs*, Zurich, 2011, pp. 78–83.
 46. Cuoq, P., Signoles, J., Baudin, P., Bonichon, R., Canet, G., Correnson, L., Monate, B., Prevosto, V., and Puccetti, A., Experience report: Ocaml for an industrial-strength static analysis framework, *Proc. 14th ACM SIGPLAN Int. Conf. on Functional Programming*, Edinburgh, 2009, pp. 281–286.
 47. Williams, N., Marre, B., Mouy, P., and Roger, M., PathCrawler: Automatic generation of path tests by combining static and dynamic analysis, *Proc. 5th European Conf. on Dependable Computing*, Budapest, 2005, pp. 281–292.

Translated by O. Pismenov